

a) write a c program to schedule the following CPU scheduling algorithms to find average turn around time and waiting time.

a) FCFS

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i;
```

```
    printf ("Enter the no. of processes: ");
```

```
    scanf ("%d", &n);
```

```
    int bt[n], wt[n], tat[n];
```

```
    for (i=0; i<n; i++) {
```

```
        printf ("Enter burst time for P%d", i+1);
```

```
        scanf ("%d", &bt[i]);
```

```
    }
```

```
    wt[0] = 0;
```

```
    for (i=1; i<n; i++)
```

```
        wt[i] = wt[i-1] + bt[i-1];
```

```
    for (i=0; i<n; i++)
```

```
        tat[i] = wt[i] + bt[i];
```

```
printf("Process |t BT |t WT |t TAT \n");  
for (i=0; i<n; i++)  
    printf("P: |d |t |d |t |d |t |d |t |d \n",  
        i+1, bt[i], wt[i], tat[i]);  
  
return 0;  
}
```

O/p.

Enter number of processes: 3

Process P1

Arrival Time: 0

Burst Time: 6

Process P2

Arrival Time: 1

Burst Time: 7

Process P3

Arrival Time: 2

Burst Time: 3

P	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	1	7	12	12	5
P3	2	3	16	14	11

Average TAT = 10.67

Average WT = 5.33

b) SJF

```
#include <stdio.h>
```

```
int main()
```

```
{
    int n, i, j, choice;
```

```
    int at[20], bt[20], ct[20], tat[20], wt[20];
```

```
    int it[20], p[20];
```

```
    float total_wt = 0, total_tat = 0;
```

```
    printf("Enter no of processes: ");
```

```
    scanf("%d", &n);
```

```
    for(i=0; i<n; i++)
```

```
    {
        printf("Process P%d\n", i+1);
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &at[i]);
```

```
printf("Burst Time: ");  
scanf("%d", &bt[i]);  
p[i] = i+1;  
rt[i] = bt[i];  
}
```

```
printf("\n1 STF Non-Preemptive");  
printf("\n2 STF Preemptive");  
printf("\nEnter your choice: ");  
scanf("%d", &choice);
```

```
if (choice == 1){  
    int temp;  
    for (i=0; i<n-1; i++){  
        for (j=i+1; j<n; j++){  
            if (bt[i] > bt[j]){  
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;  
                temp = at[i]; at[i] = at[j]; at[j] = temp;  
                temp = p[i]; p[i] = p[j]; p[j] = temp;  
            }  
        }  
    }  
    at[0] = at[0] + bt[0];  
}
```

```

for (i = 1; i <= n; i++) {
    if (ct[i-1] < at[i])
        ct[i] = at[i] + bt[i];
    else
        ct[i] = ct[i-1] + bt[i];
}
}

else if (choice == 2) {
    int time = 0, completed = 0;
    int min, shortest;

    while (completed < n) {
        min = 9999;
        shortest = -1;
        for (i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0 && ct[i] < min) {
                min = ct[i];
                shortest = i;
            }
        }

        if (shortest == -1) {
            time++;
        }
        else {
            rt[shortest]--;
            time++;
        }
    }
}

```

```

        if (rt[shortest] == 0) {
            completed++;
            ct[shortest] = time;
        }
    }
}

else {
    printf("Invalid choice!");
    return 0;
}

for (i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i];
}

printf("\n P | T A T | B T | C T | T A T | W T \n");
for (i = 0; i < n; i++) {
    printf("P %d | T %d | B %d | C %d | T %d | W %d", i+1,
        at[i], bt[i], ct[i], tat[i], wt[i]);
}

```

```
printf("Avg TAT: %.f", total-tat/n);
```

```
printf("Avg wt: %.f", total-wt/n);
```

```
return 0;
```

```
}
```

O/p.

Enter no of processes: 3

Process P1

Arrival Time: 0

Burst Time: 6

Process P2

Arrival Time: 1

Burst Time: 7

Process P3

Arrival Time: 2

Burst Time: 3

1. SJF Non-Preemptive

2. SJF Preemptive

Enter choice: 1

1/12

P	AT	DT	CT	TAT	WT
P1	2	3	5	3	0
P2	0	6	11	11	5
P3	1	7	18	17	10

Avg TAT: 10.33

Avg WT: 5.00

Comparing: for any TAT, WT, SSF non-preemptive is better.

Jan

a) Write a C program to simulate the following CPU scheduling algorithm to find TAT and WT for Priority (preemptive & Non-Preemptive).

2) ~~#include <stdio.h>~~

Priority scheduling (Non-Preemptive)

#include <stdio.h>

int main()

int n, i, j;

int at[20], bt[20], pr[20], ct[20], tat[20], wt[20],

p[20];

int temp;

float total-wt = 0, total-tat = 0;

printf("Enter no. of processes: ");

scanf("%d", &n);

for(i=0; i<n; i++)

printf("Process P%d \n", i+1);

printf("Arrival Time: ");

scanf("%d", &at[i]);

```
printf("Burst Time: ");
scanf("%d", &bt[i]);
```

```
printf("Priority (lower number → higher priority): ");
scanf("%d", &pr[i]);
```

```
p[i] = i+1;
```

```
}
```

```
for(i=0; i<n-1; i++){
```

```
for(j=0; j<n; j++){
```

```
if (pr[i] > pr[j]){
```

```
temp = pr[i]; pr[i] = pr[j]; pr[j] = temp;
```

```
temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
```

```
temp = at[i]; at[i] = at[j]; at[j] = temp;
```

```
temp = p[i]; p[i] = p[j]; p[j] = temp;
```

```
}
```

```
}
```

```
}
```

```
ct[0] = at[0] + bt[0]
```

```
for(i=1; i<n; i++){
```

```
if (ct[i-1] < at[i]{
```

```
ct[i] = at[i] + bt[i];
```

```
} else {
```

```
ct[i] = ct[i-1] + bt[i];
```

```
}
```

```
{
```

```
for (i=0; i<n; i++){
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_wt += wt[i];
    total_tat += tat[i];
}
printf("In P\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for (i=0; i<n; i++){
    printf("P%d\tAT%d\tBT%d\tPR%d\tCT%d\tTAT%d\tWT%d\n",
        p[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}
printf("Avg TAT = %.f", total_tat/n);
printf("Avg WT = %.f", total_wt/n);

return 0;
}
```

q/p: Enter no. of processes: 3

Process P1

Arrival Time: 0

Burst Time: 6

Priority: 3

Process P2

Arrival Time: 2

Burst Time: 7

Priority: 1

Process P3

Arrival Time : 5

Burst Time : 1

Priority : 2

P	AT	BT	PR	CT	TAT	WT
P2	2	8	1	10	8	0
P1	0	6	2	16	16	10
P3	5	1	2	10	5	4

Avg TAT = 9.33

Avg WT = 4.67

[Handwritten signature and date 10/3/26]

Preemptive :

```
#include <stdio.h>
```

```
int main()
```

```
{
    int n, i;
```

```
    int at[20], bt[20], pr[20];
```

```
    int rt[20], ct[20], tat[20], wt[20];
```

```
    int time = 0, completed = 0;
```

```
    int highest, min;
```

```
    float total-wt = 0, total-tat = 0;
```

```
printf("Enter no. of processes: ");
scanf("%d", &n);
```

```
for(i=0; i<n; i++) {
    printf("\n Process P%d\n", i+1);
    printf("Arrival Time: ");
    scanf("%d", &at[i]);
    printf("Burst Time: ");
    scanf("%d", &bt[i]);
    printf("Priority: ");
    scanf("%d", &pr[i]);
```

```
    rt[i] = bt[i];
```

```
}
```

```
while (completed < n) {
```

```
    min = 9999;
```

```
    highest = -1;
```

```
    for(i=0; i<n; i++) {
```

```
        if (at[i] <= time & rt[i] > 0 & pr[i] < min) {
```

```
            min = pr[i];
```

```
            highest = i;
```

```
        }
```

```
    }
```

```
    if (highest == -1) {
```

```
        time ++;
```

```
    }
```

```

else {
    rt[highest]--;
    time++;
    if (rt[highest] == 0) {
        completed++;
        ct[highest] = time;
    }
}

}

for (i=0; i<n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i];
}

printf("\nP\tAT\tBT\tPR\tCT\tTAT\tWT\n");
for (i=0; i<n; i++) {
    printf("P\t%.d\t%.d\t%.d\t%.d\t%.d\t%.d\n",
        i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i]);
}

printf("Avg TAT: %.2f\n", total_tat/n);
printf("Avg WT: %.2f\n", total_wt/n);

return 0;
}

```

1/19

O/p:

Enter no. of processes: 3

Process P1

Arrival Time: 0

Burst Time: 6 Priority: 3

Process P2

Arrival Time: 2

Burst Time: 7 Priority: 1

Process P3

Arrival Time: 5

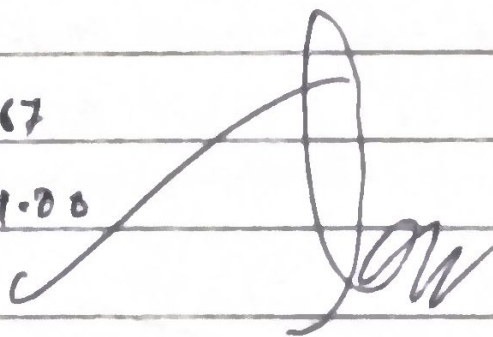
Burst Time: 1

Priority: 2

P	AT	BT	PR	CT	TAT	WT
P1	0	6	3	14	14	8
P2	2	7	1	9	7	0
P3	6	1	2	10	5	4

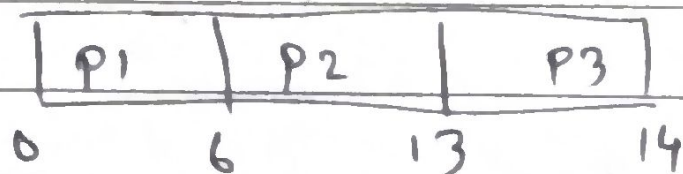
Avg TAT = 8.67

Avg WT = 4.00

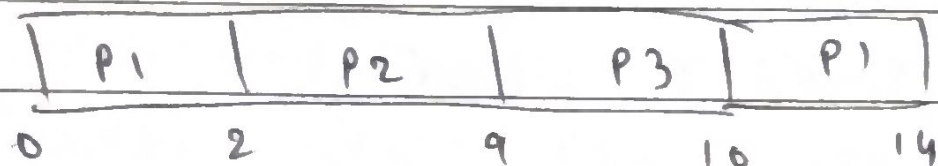
 10/3/20

1/20

Priority scheduling (Non-Pre) Gantt chart



Priority Scheduling (Pre) Gantt chart



The avg tat and wt for the preemptive case is better but normally preemptive scheduling is preferred.

Q) write a C program to simulate CPU scheduling algorithm using Round Robin (for diff. time quantum)

```
#include <stdio.h>
```

```
int main()
```

```
{
    int n, i, time = 0, remain, tq;
```

```
    int at[20], bt[20], rt[20], ct[20], tat[20], wt[20];
```

```
    float total_tat = 0, total_wt = 0;
```

```
    printf("Enter no. of processes: ");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++)
```

```
    {
        printf("\n Process %d\n", i+1);
```

```
        printf("Arrival time: ");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst time: ");
```

```
        scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i];
```

```
    }
```

```
    printf("Enter time quantum: ");
```

```
    scanf("%d", &tq);
```

```
    remain = n;
```

```

while (remain > 0) {
    int executed = 0;
    for (i = 0; i < n; i++) {
        if (rt[i] > 0 && at[i] <= time) {
            executed = 1;
            if (rt[i] > tq) {
                time += tq;
                rt[i] -= tq;
            } else {
                time += rt[i];
                rt[i] = 0;
                ct[i] = time;
                remain--;
            }
        }
    }
}

if (executed == 0) {
    time++;
}

for (i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_tat += tat[i];
    total_wt += wt[i];
}

```

```
printf("\n P\tAT\tBT\tCT\tTAT\tWT\n");
for(i=0; i<n; i++){
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
           i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}
```

```
printf("\n Avg TAT: %.2f", total_tat/n);
printf("\n Avg WT: %.2f", total_wt/n);
```

```
return 0;
```

```
}
```

o/p: Enter no. of processes: 3

Enter AT and BT for P1: 0 6

Enter AT and BT for P2: 4 4

Enter AT and BT for P3: 6 7

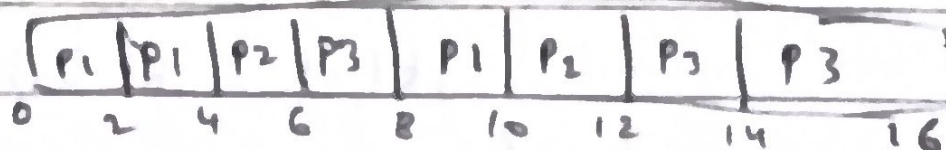
Enter time quantum: 2

P	AT	BT	CT	TAT	WT
P1	0	6	10	10	4
P2	4	4	12	9	4
P3	6	7	17	11	4

[Signature]
17/9/20

Avg TAT : 9.67

Avg WT : 4.00



Q) write a C program to simulate multi-level queue scheduling algorithm considering the following scenario.
All the processes are divided into system processes and user processes. System processes are higher priority.
Use FCFS scheduling for the processes in each queue.

→ #include <stdio.h>

```
int main() {
```

```
    int n;
```

```
    printf("total no. of processes: ");
```

```
    scanf("%d", &n);
```

```
    int pid[10], at[10], bt[10], rt[10], type[10];
```

```
    int ct[10], tat[10], wt[10];
```

```
    for (int i=0; i<n; i++) {
```

```
        printf("Process %d\n", i);
```

```
        pid[i] = i;
```

```
        printf("Arrival time: ");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst time: ");
```

```
        scanf("%d", &bt[i]);
```

```
        rt[i] = bt[i];
```

```
    }
```

```

for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (at[i] > at[j]) {
            int temp;
            temp = pid[i]; pid[i] = pid[j]; pid[j] = temp;
            temp = at[i]; at[i] = at[j]; at[j] = temp;
            temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
            temp = rt[i]; rt[i] = rt[j]; rt[j] = temp;
            temp = type[i]; type[i] = type[j]; type[j] = temp;
        }
    }
}

```

```

int sysQ[10], userQ[10];
int sf = -1, sr = -1;
int uf = -1, ur = -1;

```

```

int time = 0, completed = 0, i = 0;
int current = -1;

```

```

while (completed < n) {
    while (i < n && at[i] <= time) {
        if (type[i] == 0) {
            if (sf == -1) sf = 0;
            sysQ[++sr] = i;
        }
    }
}

```

```

else {
    if (of == -1) uf = 0;
    userA[++ur] = i;
}
i++;
}

if (current != -1) {
    if (type[current] == 1 && sf != -1) {
        userA[++ur] = current;
        current = -1;
    }
}

if (current == -1) {
    if (sf != -1) {
        current = sysA[sf];
        if (sf == sr) sf = sr = -1;
        else sf++;
    } else if (uf != -1) {
        current = userA[uf];
        if (uf == ur) uf = ur = -1;
        else uf++;
    } else {
        time++;
        continue;
    }
}
}

```

```
rt[current] --;
```

```
time++;
```

```
if (rt[current] == 0) {
```

```
    ct[current] = time;
```

```
    completed++;
```

```
    current = -1;
```

```
}
```

```
}
```

```
float total_wt = 0, total_tat = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
    tat[i] = ct[i] - at[i];
```

```
    wt[i] = tat[i] - bt[i];
```

```
    total_wt += wt[i];
```

```
    total_tat += tat[i];
```

```
}
```

```
printf("\n ID\t TYPE\t AT\t BT\t CT\t TAT\t WT\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("%.d\t %.s\t %.d\t %.d\t %.d\t %.d\t %.d\n",
```

```
        pid[i],
```

```
        (type[i] == 0) ? "system" : "user",
```

```
        at[i], bt[i], ct[i], wt[i], tat[i]);
```

```
}
```

```
printf("\n Avg WT : %.2f", totalwt/n);  
printf("\n Avg TAT: %.2f", totaltat/n);
```

```
return 0;  
}
```

o/p:

Enter no. of processes: 3

Process 0

Arrival Time: 0

Burst Time: 5

Type (0 = System, 1 = user) : 0

Process 1

Arrival Time : 2

Burst Time : 3

Type : 1

Process 2

Arrival Time : 1

Burst Time : 4

Type : 0

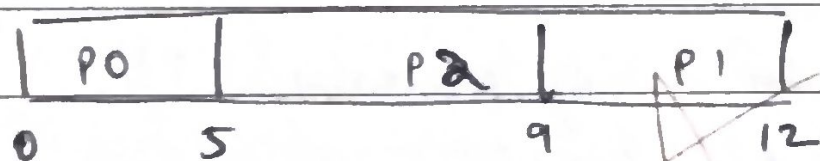
1/1/29

ID	TYPE	AT	BT	CT	TAT	WT
0	System	0	5	5	5	0
1	System	1	4	9	8	4
2	User	2	3	12	10	7

Avg WT: 3.67

Avg TAT: 7.67

Gantt chart:



24/3/26

Q) C Program to Simulate Real Time CPU scheduling
Rate Monotonic / Earliest Deadline First / Proportional

Q) Rate Monotonic & Earliest Deadline First

code :

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter no. of processes: ");
```

```
    scanf("%d", &n);
```

```
    int bt[20], dd[20], period[20];
```

```
    int ct[20], wt[20], tat[20];
```

```
    for(int i = 0; i < n; i++) {
```

```
        printf("\n Process %d: \n", i);
```

```
        printf("Burst time: ");
```

```
        scanf("%d", &bt[i]);
```

```
        printf("Period (for RMS): ");
```

```
        scanf("%d", &period[i]);
```

```
        printf("Deadline (for EDF): ");
```

```
        scanf("%d", &deadline[i]);
```

```
        ct[i] = 0;
```

// EDF

```
printf("EDF scheduling");
printf("ID\tBT\tDeadline\tCT\tWT\tTAT\n");

int time = 0;
for(int i=0; i<n; i++){
    int min-deadline = 9999, idx = -1;
    for(int j=0; j<n; j++){
        if (deadline[j] < min-deadline && ct[j] == 0){
            min-deadline = deadline[j];
            idx = j;
        }
    }

    time += bt[idx];
    ct[idx] = time;
    tat[idx] = ct[idx];
    wt[idx] = tat[idx] - bt[idx];
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
           idx, bt[idx], deadline[idx], ct[idx], wt[idx],
           tat[idx]);
}
```

// RMS

```
printf("RMS scheduling");
printf("ID\tBT\tPeriod\tCT\tWT\tTAT\n");
```

```

time = 0;
for (int i=0; i<n; i++){
    int min-period = 9999, idx = -1;
    for (int j=0; j<n; j++){
        if (period[j] < min-period && ct[j] > 0){
            min-period = period[j];
            idx = j;
        }
    }
    time += bt[idx];
    ct[idx] = time;
    tat[idx] = ct[idx];
    wt[idx] = tat[idx] - bt[idx];
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
           idx, bt[idx], period[idx], ct[idx], wt[idx], tat[idx]);
    ct[idx] = -2;
}
return 0;
}

```

__/__/33

Enter No. of Processes: 3

q/p: Process 0:

Burst Time: 3

Period (for RMS): 6

Deadline (for EDF): 10

Process 1:

Burst Time: 2

Period: 4

Deadline: 5

Process 2:

Burst Time: 1

Period: 3

Deadline: 8

EDF scheduling:

ID	BT	Deadline	CT	WT	TAT
1	2	5	2	0	2
2	1	8	3	2	3
3	3	10	6	3	6

RMS scheduling:

ID	BT	Period	CT	WT	TAT
2	1	3	1	0	1
1	2	4	3	1	3
0	3	6	6	3	6